

第23讲

量化、蒸馏与 MoE

为什么 70B 模型能用更少显存运行？为什么“总参数”不等于“每个 token 都参与计算”？

本讲主线

模型部署成本可以从不同层面降低：量化减少“每个数占多少位”；蒸馏减少“学生模型有多少参数”；MoE 减少“每个 token 实际激活多少专家”。三者不能混为一谈。



同样叫“变轻”，改变的是不同变量

学习定位：以后看到 4-bit、AWQ/GPTQ、student model、teacher logits、 $8 \times 7B$ MoE、active parameters，不再把它们当作一回事。

1. 先回答最现实的问题：模型为什么这么占显存？

最直接的第一部分，是模型权重本身。假设一个模型有 N 个参数，每个参数需要 b bytes 存储，那么仅权重的理论存储量约为：

$$\text{Weight Memory} \approx \text{Number of Parameters} \times \text{Bytes per Parameter}$$

例如 70B 参数模型，如果每个权重用 2 bytes 的 BF16/FP16：

$$70 \text{ billion} \times 2 \text{ bytes} \approx 140 \text{ GB (decimal)}$$

注意：真实推理显存还包括 KV Cache、临时 workspace、量化 scale/metadata、框架开销等。

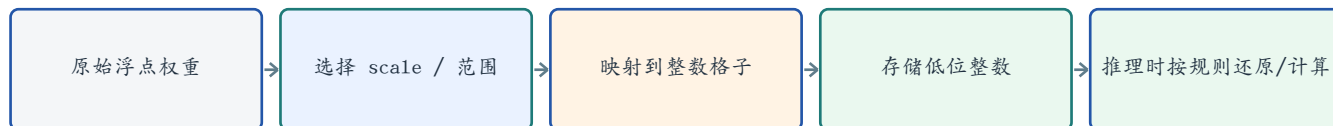
2. FP32、FP16、BF16、INT8、INT4 到底是什么？

格式	常见位宽	直觉	常见用途
FP32	32 bit	范围和精度较高，但存储大	传统训练/高精度计算
FP16	16 bit	更省显存和带宽	训练/推理；需注意数值范围
BF16	16 bit	指数范围接近 FP32，尾数更短	现代训练很常见
INT8	8 bit	离散整数级别更少	推理量化常见
INT4	4 bit	每个数可表示的级别更少	权重量化、低显存推理

位宽越低，理论存储越小，但可直接表示的数值状态也越少，因此需要设计“如何把原始浮点数映射过去”。

3. 量化 Quantization 的核心到底是什么？

量化并不是把“参数数量减少”，而是让每个参数使用更低精度的表示。最简单的均匀量化可以想象成：



例如原来权重是 0.137、0.182、0.246……，4-bit

量化不可能为每个细微小数都保留独立状态，只能把它们“吸附”到有限的离散档位。

关键

参数数量没有变：仍然是 70B 个权重。变化的是“每个权重占多少 bit，以及算子如何用这些低位表示参与计算”。

4. 一个最小量化公式：Scale 在干什么？

以对称均匀量化为直觉例子：假设 b -bit 有可用整数范围 $[-q_{\max}, q_{\max}]$ 。

$$\text{scale} = \max(|x|) / q_{\max}$$

$$q = \text{round}(x / \text{scale})$$

$$\hat{x} = q \times \text{scale}$$

$\hat{x}$ 是量化后重建的近似值。原始 x 与 $\hat{x}$ 的差，就是量化误差的一部分。

类比

像把连续的身高从“精确到毫米”改成“只有几个固定刻度的尺子”。刻度越粗，记录越省，但误差通常越大。

5. 为什么低位宽会产生误差？

因为表示级别有限。8-bit 的可用离散级别远多于 4-bit；4-bit 又远多于 2-bit。

如果一个分组中存在少数特别大的 outlier（离群权重/激活），为了覆盖它们，scale 可能被迫变大，导致大量普通值被挤在较粗的格子里。

这也是低比特量化算法为什么会研究：

- 如何识别和处理 outlier；

- 按 tensor、channel、group 分别设置 scale；

- 哪些层更敏感、是否保留较高精度；

- 如何让量化误差更贴合真实推理输入分布。

6. Per-tensor、Per-channel、Group-wise 是什么？

粒度	怎么做	优点	代价
Per-tensor	整个大张量共用一个 scale	元数据少、简单	很容易被局部 outlier 牵制
Per-channel	不同输出/输入通道各自 scale	能适应各通道分布	scale 更多、实现更复杂
Group-wise	每一小组权重一个 scale	低比特时常是精度/开销折中	多一些 metadata 和解码成本

所以“4-bit”本身并不足以说明完整方案。还要问：什么粒度？scale

怎么估？哪些张量量化？算子如何实现？

7. Weight-only Quantization 与 W8A8 有什么区别？

量化对象可以不同：

方案	权重 Weight	Activation	典型直觉
Weight-only	低位	仍保持较高精度	先解决权重存储/带宽，常见于 LLM 部署
W8A8	8-bit	8-bit	权重和激活都低精度，需更强硬件/校准支持
KV Quantization	不一定改权重	专门降低 K/V cache 精度	主要缓解长上下文/并发的 KV 显存压力

第22讲讲到的 KV Cache Quantization 与这里的 Weight Quantization 是两个不同对象。

8. PTQ 与 QAT：先训练后量化，还是带着量化训练？

方法	全称	基本流程	特点
PTQ	Post-Training Quantization	模型先正常训练好，再做校准/量化	成本低、最常见部署路线之一
QAT	Quantization-Aware Training	训练/微调时模拟量化误差	可能更能适应低位精度，但训练更复杂

很多 LLM 量化工作属于 PTQ 或 PTQ 风格，例如使用一小部分 calibration data 估计量化参数。

不要只看名字

GPTQ、AWQ、SmoothQuant 等方法关注的对象和假设不同。零基础阶段先掌握“权重/激活/校准/误差”四个维度，比死记算法名更重要。

9. 70B 模型理论权重存储：位宽有多大影响？

格式	每参数理论字节	70B 权重理论存储（十进制）	直觉
FP32	4 B	约 280 GB	最大
FP16/BF16	2 B	约 140 GB	减半
INT8	1 B	约 70 GB	再减半
INT4 理想打包	0.5 B	约 35 GB	约为 FP16 的 1/4

这里是“纯权重理论值”。真实 4-bit checkpoint/运行时通常还要保存 scale、zero-point 或其他 metadata，并且还有 KV Cache 和运行时缓冲区。

所以

“70B 4-bit $\approx$ 35GB”只能当第一数量级直觉，不能直接当作“35GB 显卡一定能完整跑起来”的承诺。

10. 为什么 4-bit 不等于一定 4 倍加速？

从 FP16 变成 INT4，权重理论存储可约降到 1/4，但端到端速度受很多因素影响：

硬件是否原生支持相应低位矩阵运算；

量化数据是否需要解包、反量化或混合精度累加；

任务当前是 compute-bound 还是 memory-bandwidth-bound；

batch size、sequence length、kernel 实现和框架调度；

KV Cache、Attention、采样等其他部分并没有都按同样比例缩小。

所以“压缩比”和“速度提升倍数”绝不能机械画等号。

11. 为什么量化模型有时几乎不掉分，有时掉得明显？

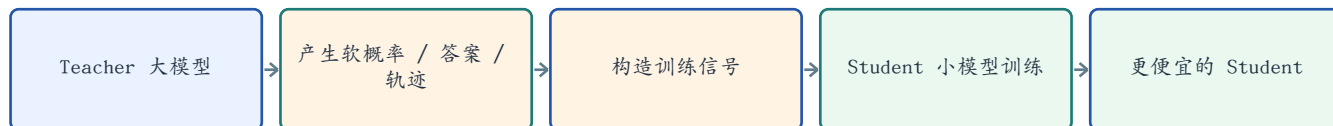
量化误差是否影响最终输出，取决于模型冗余、层敏感度、任务难度、上下文长度、量化算法和评测指标。

情况	可能现象
简单语言任务	4/8-bit 可能与高精度非常接近
数值/代码/复杂推理	低位误差可能更容易积累或放大
长上下文	权重和 KV 误差可能与位置/Attention 行为交互
只看平均 benchmark	可能掩盖少数任务、长尾样本和稳定性退化

因此研究量化模型时，除了平均准确率，还可以观察稳定性、校准、错误类型和跨 seed 方差。

12. Distillation: 蒸馏到底在“蒸”什么?

Knowledge Distillation 的目标通常是: 让一个较小的 Student 学习一个更强 Teacher 的行为或分布。



和量化不同, 蒸馏通常真的会改变模型规模: Student 可以只有 Teacher 的一部分参数。

量化: 同一套参数“每个数存得更省”; 蒸馏: 训练出“更小的另一套参数”。

13. Hard Label 与 Soft Target: Teacher 能教什么?

训练信号	Teacher 提供什么	Student 学到什么
Hard label / 最终答案	正确答案或选项	模仿任务输出
Logits / probability	完整类别/词概率分布	不仅知道哪个最对，也知道哪些备选较接近
Sequence-level response	Teacher 生成的整段答案	模仿语言风格、知识和解题模式
Reasoning / synthetic trajectories	中间步骤、工具轨迹、验证结果	尝试学习更复杂的过程策略

对现代 LLM 来说，“蒸馏”这个词经常不只指经典 logits distillation，也可能包括用强模型生成合成数据来训练小模型。要看论文具体定义。

14. 蒸馏为什么能让小模型变强？

真实标签通常只告诉 Student：“答案是 A”。Teacher 的软分布或高质量响应还可能提供：

类别之间的相似关系；

高质量语言模板；

复杂任务的分解方式；

大量人工数据中不存在的合成训练样本。

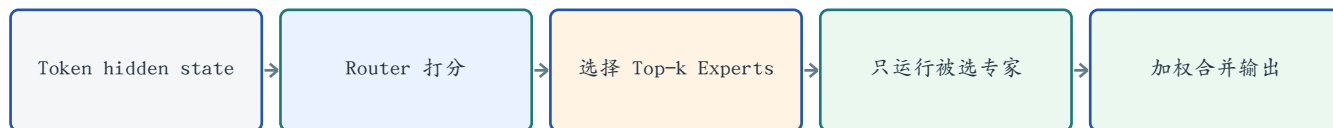
但 Student 的容量仍有限。Teacher 会的东西并不能保证全部被压缩进去。

上限不是魔法

蒸馏可以提高“小模型每个参数的利用效率”，但不是把 700B 模型无损塞进 7B。容量、数据、优化和任务分布都构成约束。

15. MoE：为什么总参数很多，但每个 token 只用一部分？

Mixture of Experts（专家混合）的核心思想是稀疏条件计算。模型中放很多 Expert，但每个 token 通过 Router 只选择少数专家。



例如有 8 个专家，但每个 token 只激活 Top-2。那么总参数可以很大，但每个 token 不需要经过全部 8 个专家。

16. Dense 与 MoE：最大的结构区别

维度	Dense Model	MoE Model
FFN / Expert	每层一个密集 FFN	某些层拥有多个 Expert FFN
每个 token	通常经过同一 FFN	Router 只激活少数 Expert
总参数	$\approx$ 每步参与计算的主体参数	总参数可远大于每 token active parameters
部署	结构相对简单	需要 Router、专家放置、跨设备通信和负载均衡

因此比较 Dense 与 MoE 时，仅看“总参数 B 数”往往会误导。

17. Total Parameters 与 Active Parameters

假设一个教学模型有：2B shared parameters + 8 个 1B experts, 每个 token 只选 Top-2。

$$\text{Total Parameters} = 2\text{B} + 8 \times 1\text{B} = 10\text{B}$$

$$\text{Toy Active Parameters per token} \approx 2\text{B} + 2 \times 1\text{B} = 4\text{B}$$

于是可以出现 “10B 总参数，但单 token 只激活约 4B” 的情况。

非常重要

Active Parameters 不是严格 FLOPs。Attention、Router、Embedding、共享层、通信、专家容量和 kernel 都有成本。它只是理解 MoE 稀疏计算的第一层直觉。

18. Router 到底干什么？

Router 通常根据当前 token 的 hidden state 对各 Expert 计算分数，再选择 Top-k。

$$\text{router_scores} = W_{\text{router}} \times h$$

然后类似：

$$\text{selected_experts} = \text{TopK}(\text{router_scores})$$

这意味着不同 token

可以走不同专家路径。专家可能逐渐出现某种功能分化，但不能仅凭名字或少量观察就断言“Expert 3 就专门负责数学”。

19. 为什么 MoE 还需要 Load Balancing?

如果 Router 发现某几个专家总是得分最高，所有 token 都挤过去，就会产生热点：

- 少数 Expert 过载，其他 Expert 闲置；

- 并行设备负载不均；

- 某些 token 可能因容量限制被丢弃或改路由；

- 专家多样性和训练稳定性可能变差。

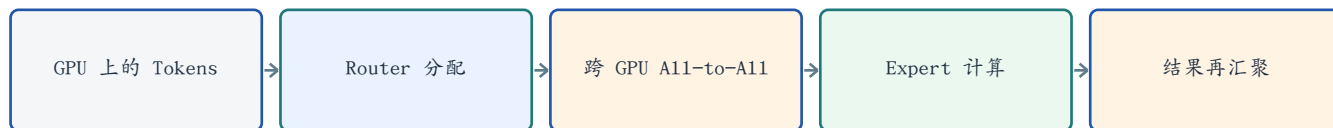
因此很多 MoE 训练会加入辅助的 load-balancing objective，鼓励流量更均衡。

类比

像 8 个窗口办业务，如果所有人都挤 2 号窗口，即使总共开了 8 个窗口，也没有获得期望的吞吐。

20. MoE 的隐藏系统成本：通信

如果不同 Expert 分布在不同 GPU 上，Router 选完专家以后，token hidden state 可能需要被发送到对应 GPU，再把结果收回来。



所以 MoE 用“稀疏计算”换来了更复杂的专家并行和网络通信。

模型层省下的 FLOPs，不一定全部转化成端到端同等比例的低延迟。

21. 量化、蒸馏、MoE：三者到底改了什么？

技术	主要改变的变量	模型参数数量	每 token 计算路径	典型目的
Quantization	每个数的表示位宽	通常不变	基本结构不变	降权重显存/带宽/部分计算成本
Distillation	训练一个更小 Student	减少	Student 自己的路径	把 Teacher 能力压进更小模型
MoE	条件稀疏激活	总参数可增大	每 token 只走少数 Expert	扩大容量，同时控制每 token 计算

它们还可以组合：例如一个 MoE 模型也可以做低比特量化；一个蒸馏得到的小模型也可以继续量化。

22. 一个现实部署模型：哪些内存一起占显存？

把第22讲和本讲合起来，推理显存至少要考虑：

Total Runtime Memory ≈ Weights + KV Cache + Temporary Workspace + Runtime Overhead	
部分	受什么影响
Weights	参数量、权重精度、量化 metadata、MoE 总专家存储
KV Cache	层数、context、batch、KV heads、KV 精度
Workspace / activations	kernel、batch、prefill/decode、实现
Runtime overhead	allocator、CUDA graph、通信 buffer、碎片等

所以“模型文件能塞进显卡”与“模型可以稳定高并发运行”不是一回事。

23. 为什么本地部署经常优先考虑量化？

对于个人电脑或单机 GPU，最先遇到的往往是显存容量。Weight-only 8-bit / 4-bit 量化可以显著降低模型权重占用，因此常成为本地部署第一选择。

但你仍要继续检查：

你的 GPU 是否有合适低比特 kernel；

最大 Context 会产生多少 KV Cache；

CPU offload / GPU offload 是否造成 PCIe 瓶颈；

量化版本在你的具体任务上是否出现明显质量损失；

吞吐、TTFT、TPOT 是否满足实际需求。

24. 对 Agent Systems 科研有什么直接影响？

以后使用不同模型做 Agent 实验时，模型“名字一样”也未必完全可比。量化等级、Serving Stack、Context、采样和硬件都可能改变行为。

混杂变量	可能造成的误读
FP16 vs 4-bit	把量化误差导致的行为差异误认为 Agent 机制差异
Dense vs MoE	只按总参数判断“模型规模”，忽略 active compute 差异
不同 Student / Distilled 版本	把蒸馏数据/Teacher 偏好当作架构效果
不同 KV precision / context	长任务稳定性与成本不可直接比较
不同 serving backend	latency/throughput 差异不一定来自模型算法

25.

一个与你长期方向很相关的问题：量化会不会改变错误相关性？

如果多个 Agent 使用同一个基础模型，但采用不同量化方式，它们可能产生一些不同输出。但这是否形成真正有价值的“异质性”，需要实验验证。

可以进一步研究：

FP16 / INT8 / INT4 的错误集合重叠有多大？

量化是否只是让准确率下降，还是会产生系统性不同的失败模式？

不同量化 Agent 组成 ensemble，是否能带来 error decorrelation？

收益能否超过直接使用同预算下更强的单模型 / Best-of-N？

科研原则

“输出变得不同”不等于“形成有用的多样性”。真正有价值的是：错误相关性下降，并且系统级收益超过额外成本。

26. 常见误区

误区	纠正
4-bit = 参数只有原来的 1/4	参数数量通常不变，是每个参数的存储位宽下降
4-bit = 一定快 4 倍	速度取决于硬件、kernel、带宽、解包和其他模块
蒸馏 = 量化	蒸馏训练更小 Student；量化降低数值精度
MoE $8 \times 7B$ = 每 token 运行 56B	每 token 通常只激活少数 Expert；但还有共享层和通信开销
Active Params = 精确 FLOPs	只是粗略理解稀疏激活，不能直接代替真实计算量
模型文件能放进 VRAM = 可以高并发运行	还要给 KV cache、workspace 和并发请求留空间

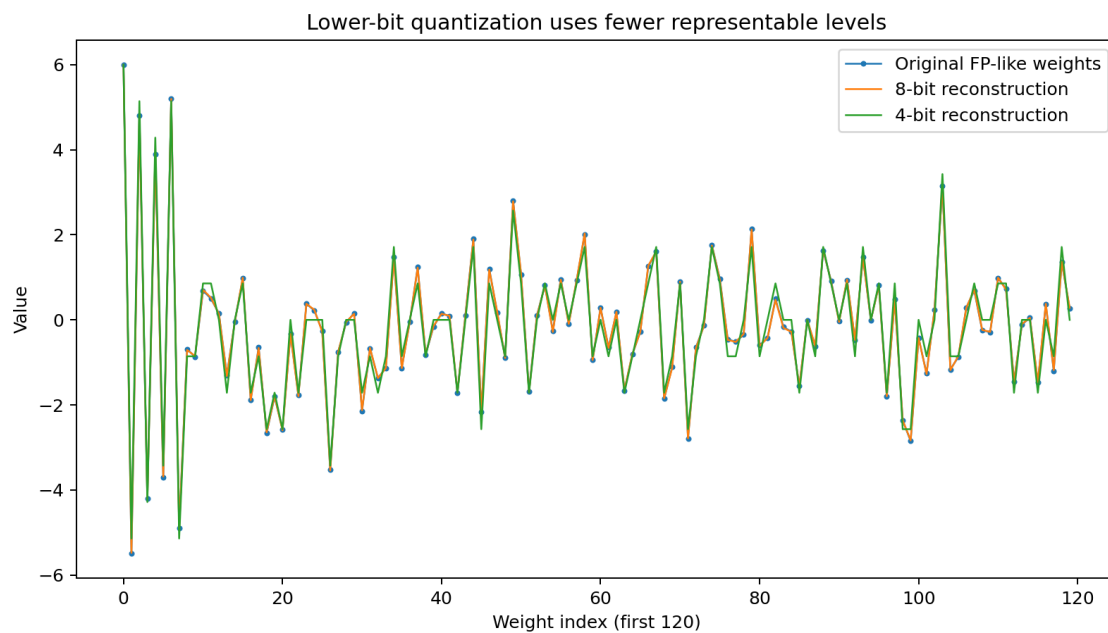
27. 小实验：亲手量化一组权重

材料包中的 `04_quantization_distillation_moe_demo.py` 使用 NumPy 做三个实验：

把同一组浮点权重分别做 8-bit、4-bit、3-bit 对称均匀量化，并计算 MSE / MAE；

计算一个 70B 参数模型在 FP32、FP16/BF16、INT8、理想 INT4 下的理论纯权重存储；

构造一个 8 Experts、Top-2 的 toy MoE，区分 total parameters 与 active parameters。



观察图时重点看：位宽越低，重建曲线越呈现明显的“离散台阶/近似误差”。

28. 如何阅读一个量化模型说明？

以后看到“XX 模型支持 4-bit”时，不要只停在 4-bit 三个字。至少继续问：

量化的是 Weight、Activation，还是 KV Cache？

是 PTQ 还是 QAT？校准数据是什么？

Per-tensor / Per-channel / Group-wise？group size 多大？

哪些层保留高精度？是否处理 outlier？

用什么硬件和 kernel 测速度？batch / context 是多少？

质量评测覆盖哪些任务，是否报告长上下文和稳定性？

这样你看到“4-bit 几乎无损”时，才知道这个结论的边界在哪里。

29. 本讲总结：把三条路线压成 9 句话

FP/BF/INT：是数值表示格式，不等于模型能力标签。

Quantization：参数数目通常不变，但每个数存得更省。

低位量化：省显存/带宽，但引入量化误差和 kernel 依赖。

PTQ：训练后量化；QAT：训练时让模型适应量化。

Distillation：Teacher 教 Student，Student 通常真的更小。

MoE：总参数可以很大，但每 token 只激活少数专家。

Total Params 与 Active Params 必须分开看。

MoE 省的是稀疏计算，但增加 Router、负载均衡和跨设备通信复杂度。

部署成本：Weights + KV Cache + Workspace + Serving/通信共同决定。

一句总括：

量化是在“数字表示”上省；蒸馏是在“模型规模”上省；MoE 是在“每个 token 激活多少计算”上省。

30. 与下一讲的连接

到这里，你已经理解了现代 LLM

的参数、训练、推理、长上下文和部署成本。下一讲开始进入一个非常关键的问题：

第24讲：Hallucination——为什么模型可以说得非常像真的，却仍然是错的？

下一讲会从 next-token objective、概率生成、知识边界、训练数据、推理和工具验证几个层面解释幻觉，并严格区分“语言流畅”“模型置信”“事实正确”这三件事。